

As you can see, the ‘refused bequest’ smell can have a real effect on the users of the software: when programmers use these two classes, they can unexpectedly get an `IllegalArgumentException`. Since JDK is a public API, it is difficult to fix this mistake, since changing the interface of these classes will result in breaking backward compatibility.

So how can one fix (i.e., refactor) this smell? Clearly, `java.sql.Date` and `java.sql.Time` require parts of the functionality from `java.util.Date`. So, if these classes use composition and contain an instance of the `java.util.Date` class, this problem could be solved.

[As an aside, it is a bad practice to use the same name for both the base and derived classes—the name of both base and derived class is `Date`, in this case. The compiler accepts it because the base and derived classes are located in different packages. To address this problem, the names of the classes `java.sql.Date` and `java.sql.Time` could be changed to `java.sql.SQLDate` and `java.sql.SQLTime`

Continued from page 58...

a URL entry in `urls.py` that maps to a function in view, which will be used to call the tasks that we have defined in `tasks.py`. Run the Celery daemon now, with the following commands:

```
python manage.py celeryd -l info -c 2 # Without CeleryBeat
python manage.py celeryd -l info -c 2 -B # With CeleryBeat
```

This is a simple method to integrate Celery with Django.

Adding multiple workers

Now scaling workers is not a concern—all you need is to ship your tasks app to a new machine, set up Celery and just start running it. The Celery daemon will start talking to the message queue, and multiple workers will start executing tasks. Celery makes sure that your task is executed once, and not by multiple workers.

Monitoring

As your application grows, so will the need to make it more stable and robust. To achieve this, you need to monitor all the components of your Celery set-up.

Monitoring RabbitMQ

To check the number of messages in your queues via the console, simply run `rabbitmqctl-status`, which will list all queues, with the number of messages in each queue. For a GUI-based output, you can simply install the RabbitMQ management plug-in.

respectively, to avoid having the same names (though in different packages.)]

Unlike ‘refused bequest’, most other smells don’t have visible or directly perceivable effects on the end software quality. However, that doesn’t mean they are not important. An analogy would be the internal cracks in a bridge or a building. These cracks may not immediately cause the structure to collapse, but if left unaddressed, they would develop further and will eventually bring down the structure. Similarly, design smells, whether their impact is externally perceivable or not, have a real impact on the quality of the software. If they are left unaddressed, they can wreak havoc and eventually result in the failure of the software project. 

By: S G Ganesh

The author works for Siemens (Corporate Research & Technologies), Bengaluru. You can reach him at sgganesh@gmail.com.

Monitoring Celery

First of all, to manage Celery, you need to switch events on—start the Celery daemon with the option `-E`, so the command would become `python manage.py celeryd -l info -c 2 -B -E`. This starts to capture events, and now you can monitor your workers, task states, etc, using:

- Celery command-line utilities
- Django-Celery admin
- Flower: A real-time Celery Web-monitor

Celery is one of the most stable systems available. It is very easy to get started with, very simple to configure, fast at executing millions of tasks, and flexible, as almost any component of Celery can be used on its own, changed, or configured as per requirements. Some other great features of Celery are:

1. *Designing workflows*: To chain multiple tasks, you can use *canvases* to divide your tasks into sub-tasks.
2. *Webhooks*: To enjoy the power of Celery using other languages like PHP, Ruby, etc.
3. *Routing*: Send tasks to a particular queue rather than any queue, and to implement all the routing mechanisms that the message broker supports.

There are loads of other great features of Celery, which are beyond the scope of this article. I am sure that if you have a use-case, chances are that you can do it with Celery. 

By: Konark K. Modi

The author works as a senior systems engineer with Website Operations at Makemytrip.com, and is a technology enthusiast at heart. Follow him at: [@konarkmodi](https://twitter.com/konarkmodi).